

Programação de Sistemas

Alocação dinâmica de memória em C

Marcelo de Gomensoro Malheiros



Recapitulando...

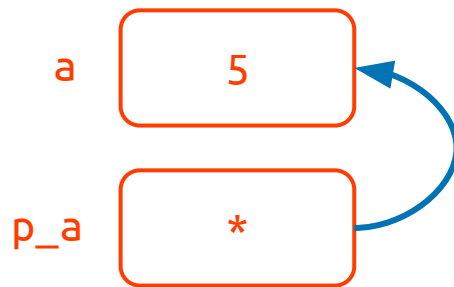
» Ponteiros



- Ponteiro é simplesmente uma referência para outra variável
 - Um ponteiro é **declarado** usando ***** junto ao tipo para que vai apontar
 - Ele deve ser **inicializado** com **&**, apontando para uma variável já existente

- Exemplo:

- `int a = 5;`
- `int *p_a = &a;`

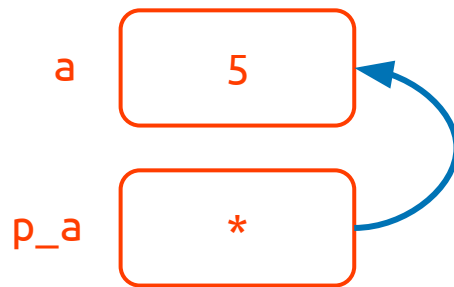


Recapitulando...

» Ponteiros



- O valor apontado pelo ponteiro pode ser acessado usando *****
 - O ponteiro **tem** que apontar para uma variável
 - Recomenda-se o formato **(*p)** para não confundir com o operador de multiplicação



- Exemplo:
 - `int a = 5;`
 - `int *p_a = &a;`
 - `int b = (*p_a);`
 - `printf("%d\n", (*p_a));`

Recapitulando...

» Ponteiros e *strings*



- Uma *string* podem ser representada como ponteiro para *char*
 - Funciona da mesma forma que um vetor de caracteres
- Exemplo:
 - `void func(char s1[], char *s2) {`
 - `printf("%s %s\n", s1, s2);`
 - `}`
 - `int main() {`
 - `char b[] = "bola";`
 - `char *c = "dado";`
 - `func(b, c);`

Alocação dinâmica

- Função que retorna o **número de bytes** ocupado por um tipo
 - retorna um valor “inteiro grande”, ou **unsigned long**
 - se necessário, pode ser convertido para inteiro simples com **(int)**
- Exemplo:
 - `printf("bytes int: %lu\n", sizeof(int));`
 - `printf("bytes float: %i\n", (int) sizeof(float));`

- Reserva uma área **não inicializada** com o tamanho em **bytes**
 - é preciso importar esta função usando `#include <stdlib.h>`
 - o valor retornado é um **ponteiro** para o **início** da área de memória criada
- Exemplo:
 - `#include <stdlib.h>`
 - `int main() {`
 - `int *p_i = malloc(10 * sizeof(int));`
 - `for (int c = 0; c < 10; c++) {`
 - `p_i[c] = 0;`
 - `}`

- Libera a área de memória vinculada a um ponteiro
 - o programador deve chamar **explicitamente** quando não for mais usada
 - o ponteiro **não pode ser mais usado** depois de liberado!
- Exemplo:
 - `#include <stdlib.h>`
 - `int main() {`
 - `int *p_i = malloc(10 * sizeof(int));`
 - `// ...`
 - `free(p_i);` // p_i não pode ser usado daqui em diante
 - `p_i = NULL;` // recomenda-se zerar o ponteiro em seguida

Retorno de vetor alocado dinamicamente



» Exemplo de função

- Vetores só podem ser retornados de uma função se tiverem sido alocados **dinamicamente**
 - `int *cria_vetor(int tamanho) {`
 - `int *p_i = malloc(tamanho * sizeof(int)); // alocação dinâmica`
 - `// inicializa o vetor`
 - `for (int c = 0; c < tamanho; c++) {`
 - `p_i[c] = 1;`
 - `}`
 - `return p_i; // retorna o vetor`
 - `}`

Retorno de vetor alocado dinamicamente



» Exemplo de chamada

- A função retorna um ponteiro, que será usado normalmente:

- `int main() {`
- `int *p_z = cria_vetor(10);`
- `p_z[9] = 222;`
- `for (int j = 0; j < 10; j++) {`
- `printf("[%i] = %i\n", j, p_z[j]);`
- `}`
- `free(p_z);`
- `p_z = NULL;`
- `return 0;`
- `}`

Exercícios

- Use alocação dinâmica para construir a *string* resultante para a concatenação das *strings* “copa” e “cabana”. Não esqueça do terminador.
- Crie uma função que recebe uma *string* e que devolve uma nova *string*, alocada dinamicamente, com todas as vogais removidas. Teste esta função passando duas *strings* distintas.

Perguntas?